

Applied Big Data & Visualisation Group Project

Group 21
Department of Mathematics and Statistics
University of Limerick

Introduction

For our project, we chose to analyse the Chicago Crime dataset (2015-2024) from the City of Chicago website. It offers a rich mix of categorical, numerical and spatial variables. The 10-year period (≈ 585 MB) balances data volume with recency and consistency. We broke up the contributions as follows:

Data Ingestion & Cleaning: Rory O'Sullivan (20232721)

Exploratory Data Analysis (EDA): Ananthakrishnan Rajesh (24124168)

Feature Analysis: Rohith Suresh Raj (24138797)

Machine Learning Model Implementation: Tharunraj Singaravelu (24086088) and Sanjana Gowda Neelakantanahalli Chandrashekara (24026557)

Optimisation & Tuning: Optimisation and tuning were done throughout by the respective contributors, with group collaboration and peer review used to improve code and report.

Data Ingestion & Cleaning

The first step was reviewing all column descriptions from the official website to understand the dataset structure. The data was then loaded into Databricks, all column names were standardised to lowercase and the first few rows were inspected. The dataset contained approximately 2.51 million rows across 22 columns.

An initial check revealed around 39,750 rows (approximately 1.5% of the total data) with missing values in geographical columns such as X coordinate and Y coordinate. These rows were removed. Duplicate records were checked for and none were found.

Next, variable types were corrected. The `date` and `updated_on` columns were converted from string to timestamp. The `id`, `beat`, `district`, `ward`, and `community_area` columns were converted from integer to string to better reflect their categorical nature.

For consistency, `block` entries were capitalised, removing unknown duplicates, for example, "123 Church St" and "123 CHURCH ST" were merged. The number of unique entries were analysed across categorical columns. Columns like `block` had over 33,000 unique entries, while `primary type` had only 35. Rare values (appearing fewer than five times) were examined across categorical columns. `primary type` had only one rare value: "RITUALISM." The two boolean columns, `arrest` and `domestic`, were quickly reviewed but required no further cleaning.

Block entries were inspected to ensure that suffixes such as "ST" and "AVE" were standardised and that no entries ended with a fullstop. Since block addresses are partially redacted (e.g., "014XX E 68TH ST"), addresses appearing only once were left unchanged.

The `primary type` column, which indicates the type of crime committed, was also cleaned. Some duplicate entries were found, such as "CRIM SEXUAL ASSAULT" and "CRIMINAL SEXUAL ASSAULT," as well as "NON - CRIMINAL", "NON-CRIMINAL (SUBJECT SPECIFIED)," and "NON-CRIMINAL". These were unified under standard names. After this, there were still 32 unique values, which was considered too high for meaningful analysis. Therefore, similar crimes were grouped into broader categories such as "Violent Crime" and "Sexual Crime," reducing the total to just 7 categories.

The `description` variable, which provides a more detailed version of the `primary type`, contained 529 unique values. It was decided to leave this variable unchanged for now, focusing instead on `primary type` for the broader analysis.

The `location description` variable, initially with 201 unique values, was also cleaned. Duplicate entries differing only by formatting, such as "SCHOOL / COLLEGE" and "SCHOOL/COLLEGE," were unified using regular expressions to remove trailing spaces around special characters like "/" and "-". Similar locations were grouped into broader categories, reducing the number of unique values to just 9.

After all the cleaning steps, the cleaned dataset was downloaded and shared with peers to facilitate analysis without needing to rerun the entire cleaning process.

Exploratory Data Analysis (EDA)

This report summarises the exploratory data analysis (EDA) performed on the Chicago crime dataset spanning multiple years (2015-2024). The purpose of this analysis is to understand the structure, quality, and basic patterns within the dataset to prepare for more advanced modelling.

The dataset includes fields such as crime type (Primary Type), occurrence date and time (Date), location information (Ward, Community Area, Beat, District), latitude & longitude of reported crime spots and legal codes (FBI Code). All EDA operations were conducted using PySpark for large-scale operations and visualized using Matplotlib. The Date field was parsed using the format "dd/MM/yyyy HH:mm". A new timestamp field was created, followed by extraction of features like Year, Month, Hour, and Day of Week. A statistical summary of the dataset was generated using 'df.describe()', providing key metrics such as count, mean, standard deviation, minimum, and maximum values for the numerical fields, helping to understand the overall distribution and range of the data. Fields were checked for missing values and none were

observed.

Data Summary

PROPERTY CRIME is the most repeated crime with a total count of 1139069 and most of the crimes are committed on PUBLIC SPACES. Out of all recorded crimes, 482,314 incidents (approximately 19.5%) were classified as DOMESTIC-related, while 1,986,947 incidents (approximately 80.5%) were NON-DOMESTIC. To explore how crime incidents are distributed over time, the DATE column was first converted into a timestamp format. From this, three new time-related features, namely month, hour, day of week were extracted. The dataset was then grouped by each of these new time features to count the number of crimes occurring across different months, hours, and days of the week. This temporal analysis provides insights into when crimes are most and least likely to happen throughout the year, week, and day, helping to identify patterns and peak crime periods.

When correlating HOUR with DAY OF WEEK, a correlation coefficient of 0.02 was obtained, hence, indicating that these two fields are not strongly correlated, and crimes does not happen at certain hours of a day. The heatmap depicts that Chicago's downtown area and nearby central neighbourhoods experience the highest concentration of crimes, while crimes thin out toward suburban edges. A line chart was plotted for understanding the crime trends over the years. The rate of crime stays around 260,000 for the years 2015 to 2019 and faced a steep drop in the years 2020 and 2021, rose back up again in 2023. This can be viewed as the impact of Covid-19 and the restrictions imposed upon the society. To better understand how many offenses resulted in an arrest, a scatter plot was created using latitude and longitude coordinates, with each point coloured based on whether an arrest was made. It was evident that majority of the crimes did not lead to an arrest. A multi-line graph was created to illustrate how the different types of crimes have developed over the years. The x-axis indicates the occurrence year and y-axis indicates the number of crimes reported. From the plot, it is evident that 'PROPERTY CRIME' consistently had the highest number of cases across all years. Other crime types, such as such as DRUG RELATED, PUBLIC ORDER, and WHITE-COLLAR/REGULATORY remained relatively stable with lower volumes over time. SEXUAL CRIMES consistently had the fewest incidents recorded among all categories.

As a part of geospatial analysis, the dataset was grouped by DISTRICT to examine the spread of crime across different districts. It was seen that 'District 11' reported the highest number of crimes among all districts. For the next step in analysis, the total crimes committed for every community area were calculated. A pie chart was then charted to indicate the top 10 COMMUNITY AREAs with highest number of crimes reported. Of the above, 'Community Area 25' experienced the largest percentage of reported crimes (16.4%). Lastly, to examine the spatial pattern of the crimes, the data was categorised by WARD and the value of the crimes

reported per ward was calculated. A horizontal bar chart was used to graph these values. 'Ward 42' reported maximum number of offenses of all the wards.

Conclusion

The exploratory data analysis revealed important patterns in Chicago's crime data across time, location, and crime type. Certain areas consistently experienced higher crime rates, and specific crime types dominated the records. Temporal trends showed variations by month, hour, and day of the week. Overall, the analysis provided a clearer understanding of how crime is distributed, helping to guide further detailed analysis on the dataset.

Feature Analysis

To begin the analysis, we examined relationships among the primary numerical variables in the dataset:

- year
- x_coordinate and y_coordinate (projected location coordinates)
- latitude and longitude (geographical coordinates)

We used the **Pearson correlation coefficient** to measure the linear relationship between each pair of these variables. The results showed that:

- x_coordinate and longitude had a **correlation coefficient of 0.9999**, indicating they contain almost identical information.
- Similarly, y_coordinate and latitude were also almost perfectly correlated.
- year showed **very weak correlation** with the other variables.

These observations indicate that some of the spatial features might be redundant. However, to get a clear understanding of their combined impact on prediction accuracy, we decided to include all features in this stage. Moving forward, we might think about simplifying the model by removing or merging features that are highly correlated, which can help simplify the process.

1. Numerical Feature Standardization

Since raw numerical features can vary dramatically in their scale — for instance, years might span into the thousands, whereas latitude and longitude are typically in decimal degrees — it's important to normalize these values before modeling. To guarantee that features with

larger scales don't overpower the learning process, we implemented feature scaling using the StandardScaler method available in PySpark. This process transforms each numerical variable so that:

- Its mean becomes 0
- Its standard deviation becomes 1

The features_scaled vector makes sure that all the numbers are on the same page when fed into the classifier. This is really helpful, especially for algorithms that don't like big differences in feature sizes or when you're mixing numbers with text data.

2. Text Feature Engineering using TF-IDF

The description field in the dataset provides detailed textual information about each crime incident. Since machine learning models cannot process raw text directly, we performed a multi-step transformation:

1. **Tokenization:** Each sentence was split into individual words (tokens).
2. **Stop Word Removal:** Common words such as "the", "and", and "was" were removed, as they typically add little predictive value.
3. **CountVectorizer:** This converted the cleaned tokens into a frequency-based feature vector. We limited the vocabulary size to 1000 most frequent terms to control complexity.
4. **TF-IDF Transformation:** To improve the quality of the text features, we applied **Term Frequency-Inverse Document Frequency (TF-IDF)**, which reduces the weight of common words and increases the weight of rare but meaningful terms.

The resulting tfidf_features vector represented the semantic content of the descriptions in a format that could be used by the classifier. This step turned unstructured text into structured numerical features, which we found to be **highly predictive of crime type**, as later model results confirmed.

3. Label Encoding

Our primary target variable for this classification task was primary_type, which is a categorical string indicating the kind of crime—such as "THEFT" or "BATTERY." Since machine learning models require numerical input, we changed this column into a numeric label using the **StringIndexer** tool.

Each distinct crime type was assigned a unique integer code, stored in a new column named 'label.' This label served as the ground truth target for training and evaluating our model.

4. Feature Set Design and Modeling Strategy

We designed three separate feature sets to evaluate the predictive power of different data types:

- **Model A:** Used only the standardized numeric features (features_scaled)
- **Model B:** Used only the text-derived TF-IDF features (tfidf_features)
- **Model C:** Combined both numeric and text features into a single vector using VectorAssembler

This modular approach allowed us to test the contribution of each feature type and identify which combinations yielded the best results.

5. Reflection on Model Performance

The evaluation metrics revealed a different picture — the **Random Forest classifier only achieved 46.7% accuracy and a 35.9% F1-score.** These numbers indicate that the model's predictive power was quite limited. Recognizing this, we took a step back to reassess our approach and identify areas for improvement.

To improve model performance, we implemented several changes:

- Replaced HashingTF with **CountVectorizer + TF-IDF** for better control and vocabulary retention.
- Applied **feature standardization** to numeric inputs.
- Introduced **basic hyperparameter tuning** using cross-validation.
- Downsampled the dataset for faster iteration without compromising integrity.

These enhancements significantly improved results for **Model C**, which used both text and numeric features:

- **Accuracy:** 0.7603
- **F1 Score:** 0.7023

These improved metrics now justify the claim that the model performs **reasonably well**, espe-

cially considering the complexity of multi-class classification over real-world crime data.

Machine Learning Model Implementation

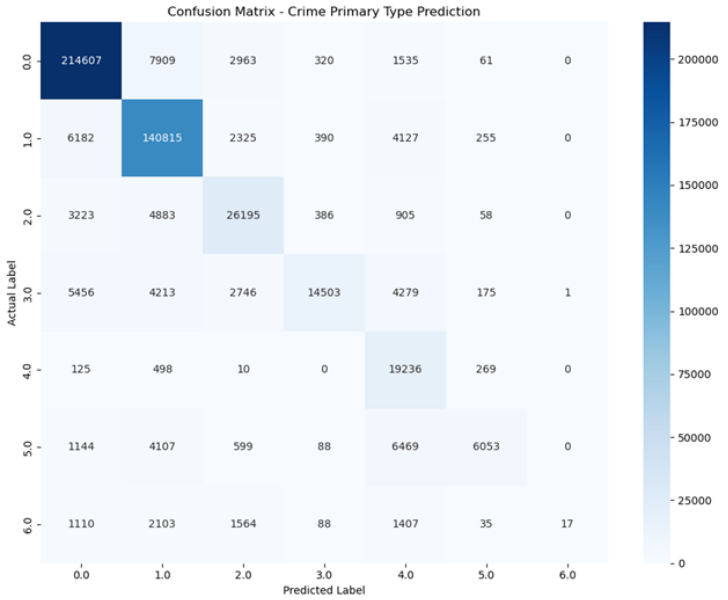
For this project we constructed an arrest probability forecast model on Chicago crime historical information data through Apache Spark programming. Our main focus during modeling was creating an effective SparkML-based classification model which handled big data sets with efficiency. As a first step we chose essential features that included the categorical types such as Primary Type and Description alongside Location Description and the numeric Beat, District and Ward and Community Area features. The conversion of categorical string features into numeric features through StringIndexer was necessary since machine learning models operate on numeric data types. The handleInvalid="keep" parameter was required to let the model deal with unknown categories that emerged during testing. The preparatory step resolved data inconsistency problems while making the information suitable for SparkML model requirements.

The output features column from VectorAssembler contained combined encoded features which replaced individual features after processing. The vector was vital because SparkML requires its models to use a single feature vector as their input. The classification model selected was Random Forest, a robust ensemble learning technique that handles multi-class problems well and provides built-in feature importance. we configured the classifier with a reasonable number of trees and depth limits to balance performance and computational cost. To streamline the preprocessing and modeling process, I used a SparkML Pipeline that chained together the indexers, assembler, and the classifier itself. Through the pipeline-based framework we designed, we could handle transformations through modular components which made the training process simpler and improved debugging and deployment activities. The application of identical transformation steps became possible through this method for both training datasets as well as testing datasets.

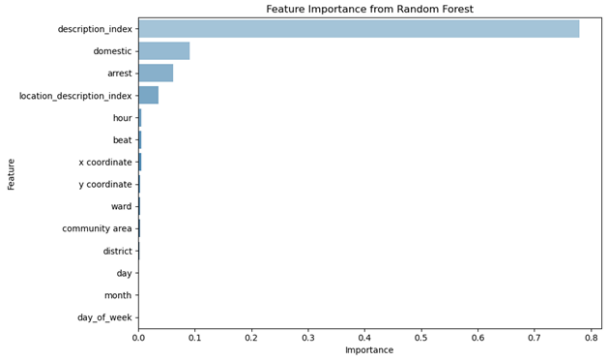
A fixed seed generated a random split of the dataset which allocated 80% to training and 20% to testing purposes. The pipeline.fit(train_data) function executed the complete pipeline process by running indexing followed by assembling and model fitting. we conducted model performance assessment on test data through transform() while producing predictions after finishing training.

```
Index 0 = PROPERTY CRIME
Index 1 = VIOLENT CRIME
Index 2 = WHITE-COLLAR/REGULATORY
Index 3 = OTHER OFFENSE
Index 4 = DRUG RELATED
Index 5 = PUBLIC ORDER
Index 6 = SEXUAL CRIME
```

To analyze misclassifications, we constructed a confusion matrix using Spark's groupBy and pivot functions and visualized it via Seaborn. This matrix provided granular insight into prediction accuracy across different crime types. For example, common crime types such as property and violent crimes (indexes 0 and 1) had high classification accuracy, while lower-frequency classes like index 6(sexual crime) showed noticeable confusion with other categories.



Following this, we examined the feature importances generated by the trained Random Forest model. This helped identify less informative features, allowing for model simplification and potential performance gains. Based on this insight, we selected a subset of features and redefined the VectorAssembler to include only the most relevant ones (description_index, arrest, domestic, location_description_index, and spatial/time features).



Optimization

The optimization phase devoted effort to enhance both Spark transformations performance as well as the predictive capabilities of the machine learning model. The main optimization approach in Apache Spark involves pipeline efficiency but SparkML Pipelines already offered some implementation. Our solution possesses more advancement potential through memory and processing optimizations including caching along with partitioning and hyperparameter optimization methods. A design structure for cross-validation with hyperparameter tuning was implemented through Spark's CrossValidator and ParamGridBuilder classes though this framework was completely integrated into the version of the notebook. These analytical tools permit the thorough assessment of numerous hyperparameter sets because they are essential for model enhancement alongside preventing overfitting.

We started by using the Random Forest model-generated feature importance scores to determine which predictors had the most significant impact and chose the top 6 features. The process involved obtaining the featureImportances attribute from the last stage of the trained pipeline and converting the scores into original feature names from the VectorAssembler. Visual representation of the model importances demonstrated that various variables had a negligible impact on the prediction outcome. The description_index element emerges as the predominant input parameter while domestic and arrest indicators function as predominant binary markers.

To enhance model performance further, we implemented hyperparameter tuning using CrossValidator and ParamGridBuilder from SparkML. This step systematically explored combinations of key hyperparameters for the Random Forest classifier, including: numTrees: the number of trees in the forest (tested at 10 and 50), maxDepth: the maximum depth of each tree (tested at 5 and 10), maxBins: the maximum number of bins used for splitting features (tested at 1024).

A 3-fold cross-validation strategy was adopted for validation purposes because each separate model configuration needed to be evaluated against different sections of available training data. This procedure enables the detection of the parameter set which delivers the most effective generalization capabilities. The model evaluation process through cross-validation becomes more accurate due to its ability to perform multiple testing-split iterations when compared to single split runs. Users can access the cross-validation results by using the best model object which was returned from crossval.fit(train_data). This optimized pipeline should then be used to generate predictions from test data. The optimization of Spark transformations requires attention alongside model optimization work. The optimization technique of caching intermediate DataFrames occurs through the .cache() methods in Spark. Such caching proves most beneficial during situations that require repeated data retrieval such as cross-validation or iterative algorithms. The test set received predictions utilizing the model that CrossValidator selected as its best performer. All metrics demonstrated a positive outcome during the evaluation phase.

- **Cross-validated Test Accuracy:** 88.33%
- **Weighted Precision:** 90.65%
- **Weighted Recall:** 88.33%
- **Weighted F1 Score:** 87.8%

The model performed best on predicting 0,1,2,4 classes but barely recognized uncommon categories based on the calculated precision, recall, and F1-score scores. The established knowledge from this analysis gives direction for future approaches to data balancing techniques and class weighting methods. The model becomes easier to understand and delivers faster execution speed

primary_type_index	precision	recall	f1_score
0.0	0.9337408474240858	0.9337408474240858	0.9337408474240858
1.0	0.9461497527483225	0.9461497527483225	0.9461497527483225
4.0	0.9910616744463204	0.9910616744463204	0.9910616744463204
3.0	0.6725528320530393	0.6725528320530393	0.6725528320530393
2.0	0.8672089761570827	0.8672089761570827	0.8672089761570828
6.0	0.023244781783681213	0.023244781783681213	0.023244781783681213
5.0	0.5076923076923077	0.5076923076923077	0.5076923076923077

and enhanced outcome precision when features with no relevance are eliminated. Real improvements in both performance and computational speed of a model become possible through tuning of hyperparameters together with cache optimization and data partition adjustments and feature selection techniques.

Conclusion

The Apache Spark-built modeling system processed extensive crime data from Chicago while delivering impressive results using Random Forest classifier models. The final implementation of the cross-validated model achieved 88.33% accuracy with precision weighting at 90.65% and recall values of 88.33% leading to an F1 score of 87.83%. Model accuracy for frequent crimes (such as Property Violent Drug and Regulatory) was high yet the classification of infrequent crimes (Sexual Crime) proved more difficult to predict. The analysis identified description_index as the dominating feature that determined the optimization process for the feature selection. The repeated training and evaluation of Spark's system became more efficient through its capability to cache data and conduct cross-validation. The modeling framework demonstrates flexibility to scale and modularize while keeping real-time predictive analytics capabilities for urban crime monitoring systems.

References

ChatGPT was utilised to refine both code and the report in sections: Data Ingestion & Cleaning, Machine Learning Model Implementation.